

TECHCATALYST DE 2026

Python foundations

Variables to functions, written by hand. By 5pm you'll read
tracebacks without flinching.

Week 2 Day 2

Types & variables

Lists, dicts, tuples, sets

Control flow & loops

Functions

Reading tracebacks

Today's agenda

Time	Block
8:00 to 8:30	Recap + stand-up
8:30 to 9:30	Types, variables, strings, live coding, you type along
9:45 to 11:00	Collections: lists, tuples, dicts, sets
11:00 to 12:00	Drills block 1 (variables, conditionals)
1:00 to 2:00	Control flow & loops
2:00 to 3:00	Functions
3:15 to 4:30	Drills block 2 (lists, loops, dicts, functions)
4:30 to 5:00	Traceback clinic + recap

How today works

I type, you type

Live coding all day, keep a notebook open and type every example

Typos are the curriculum: we debug them together

No copy-paste. Fingers learn syntax, eyes don't

Why so basic?

Week 4 you'll write dbt models; Week 5, PySpark, both assume this fluency

Most pipeline bugs are Python bugs: types, None, off-by-one

The drills are from past cohorts, they work

If you already know Python: the drills have Challenge tiers, and you'll be drafted as a floating helper.

Variables, types, strings

```
trip_count=3_421_809    # int (underscores for readability)
avg_fare = 18.42        # float
vendor= "CM T"          # str
is_weekend = False      # bool
tip = None              # no value yet!, not zero!
```

```
# f-strings: how you'll format everything
print(f'{vendor}: {trip_count:,} trips, avg ${avg_fare:2f}')
CM T:3,421,809 trips, avg $18.42
```

DE angle: type confusion is the #1 source of dirty data, "18.42" (string) is not 18.42 (float). type() and casting are your first debugging tools.

Collections: pick the right container

Container	Syntax	Ordered?	Mutable?	DE use case
list	[1, 2, 3]	Yes	Yes	Rows, batches, anything sequential
tuple	(40.7, -74.0)	Yes	No	Fixed records: (lat, lon)
dict	{"id": 7}	Yes*	Yes	JSON records, lookups, configs
set	{"CMT", "VTS"}	No	Yes	Dedup, membership tests

*dicts keep insertion order (3.7+). Rule of thumb: JSON from an API to dicts and lists, all the way down.

Working with dicts (your JSON future)

```
trip = {"vendor": "CM T", "fare": 18.5, "zone": "JFK"}

trip["fare"]          # 18.5, KeyError if missing
trip.get("tip", 0.0)   # 0.0, safe default, no crash
trip["tip"] = 3.25     # add / update a key

for key, value in trip.items():
    print(f"{key} = {value}")

zones = [t["zone"] for t in trips] # extract one field from many records
```

.get() with a default is the difference between a pipeline that runs and one that dies at 2am.

Control flow

```
fare = 412.00

if fare < 0:
    label= "invalid"
elif fare > 300:
    label= "outlier"      # this branch wins
else:
    label= "normal"
```

```
# truthiness shortcuts
if not trips:          # empty list
    print("nothing to bad")

if vendor in {"CM T", "VTS"}:
    process(trip)
```

Indentation IS the syntax, 4 spaces, consistently. Your editor handles it; your job is to notice when blocks are wrong.

Loops: for, range, enumerate, zip

```
fares = [12.5, 8.0, 31.2, 9.9]

total = 0
for fare in fares:      # iterate values directly
    total += fare

for i, fare in enumerate(fares):  # need the index too?
    print(f"row {i}: {fare}")

for fare, zone in zip(fares, zones):  # two lists in lockstep
    print(f"{zone}: {fare}")

while retries < 3:      # when you don't know how many times
    retries += 1
```


Functions: name your logic

```
def clean_fare(raw , m ax_fare=300.0):  
    """Convert a raw fare string to float; None if invalid."""  
    try:  
        fare = float(raw )  
    except ValueError:  
        return None  
    if fare < 0 or fare > m ax_fare:  
        return None  
    return round (fare,2)  
  
clean_fare("18.42")    # 18.42  
clean_fare("N/A")      # None  
clean_fare("950",m ax_fare=1000) # 950.0 , keyword override
```

Small, single-purpose, testable functions, tomorrow we'll see this exact one again with error handling.

Reading a traceback (bottom-up!)

```
Traceback (most recent call last):  
  File "load.py", line 12, in <module>  
    total += trip["tip"]  
  File "load.py", line 12 ...  
KeyError: 'tip'
```

- Read the **LAST** line first: the error type and message.
- Then walk up to find YOUR file and line number, ignore library internals at first.
- Then ask: what value made this happen? `print()` or breakpoint to confirm.
- Today's clinic: we'll dissect the 6 errors you'll hit most (KeyError, TypeError, IndexError, NameError, ValueError, IndentationError).

Drills block 1, variables & conditionals

Goal, build fluency through repetition: small, fast exercises, no AI, no copy-paste.

60 min

1. Work through 01-Variables, 02-Conditionals, 03-RockPaperScissors in Labs/Day 2/Python Drills.
2. Solo first (30 min), then compare with your table and reconcile differences.
3. Stuck >5 min? Write down the error, try one fix, then ask your table.
4. Finished? Do the Challenge variants, then float and help.

Deliverable: Completed drill scripts committed to your repo under week2/day2/.

Drills block 2, lists, loops, dicts, functions

Goal, the same fluency loop, now over collections and functions.

75 min

1. Complete 05-Lists, 04-Loops, 06-Dictionaries, 08-Functions, 09-Nesting.
2. 09-Nesting is the boss level: a dict of lists of dicts, exactly what APIs return (tomorrow!).
3. Same protocol: solo 40 min, reconcile at the table, commit.

Deliverable: Drill solutions committed + one "aha" note in your README about something that surprised you.

Key takeaways

- 1 Types first: most dirty data is type confusion, and `type()`/casting are debugging tools.
- 2 Containers have jobs, lists for sequences, dicts for records, tuples for fixed shapes, sets for dedup.
- 3 Functions: small, single-purpose, with defaults and docstrings.
- 4 Tracebacks read bottom-up: error message to your line to the value that caused it.
- 5 Tomorrow: error handling, files, classes, and your first real API calls.